I. Question (Heaps)

Given index of a d-heap node and a level l, write a recursive function that returns the number of descendants at level l of that heap node. Level 1 corresponds to children, Level 2 corresponds grandchildren, Level 3 corresponds grand grand children of that heap node.

```
1 int descendants(int no, int level)
```

II. Question (Heaps)

Given an array of N integers, find the k'th maximum of those integers in $\mathcal{O}(N \log K)$ time. (Hint: Use a min-heap to store K largest elements at a time, in that case the removeMin will return the k'th maximum).

```
1 int kthMaximum(int[] array, int k)
```

III. Question (Disjoint Sets)

Write a function that merges three sets given their indexes. You can use *findSet* method, but not the original *union* method. Merge the sets such that the resulting merged set will have the minimum depth. Update also the depth if needed.

```
1 void union(int index1, int index2, int index3)
```

IV. Question (Disjoint Sets)

You are given a set of equalities such as

```
0=9
1=2
3=5
5=7
9=4
5=4
6=8
```

where numbers correspond to variables. When the equalities are combined, we get

```
0=9=4=3=5=7
1=2
6=8
```

3 equalities. Write the function that finds the number of equalities when combined

```
1 int combine(int N, int [] left , int [] right)
```

where N represents the number of variables, left and right represent the left and right parts of the equalities.

V. Question (Graphs)

Write a method that checks if the graph is complete bipartite graph or not. Write the function for adjacency matrix representation. A graph (V_1, V_2) is said to be a complete bipartite graph if every vertex in V_1 is connected to every vertex of V_2.

```
1 boolean isCompleteBipartite()
```

VI. Question (Graphs)

Write a function that computes the number of bidirectional edges in a graph. Write the function for adjacency list representation.

```
int bidirectionalEdges()
```